

MyMavenGuavaApp

MyMavenGuavaApp – Simple Maven Project Example

MyMavenGuavaApp is a simple Java project created using Apache Maven and the Google Guava library.

This project demonstrates how Maven automatically downloads dependencies and builds Java applications.

Steps to execute the project:

Step1:

To create a MyMavenGuavaApp project, run the following command:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
vaishnavi@vaishnavi-virtualbox:~/devops21$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ----- org.apache.maven:standalone-pom -----
[INFO] Building Maven Stub Project (No POM) 1
[INFO]
[INFO] ----- [ pom ] -----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/vaishnavi/devops21
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenGuavaApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Project created from Old (1.x) Archetype in dir: /home/vaishnavi/devops21/MyMavenGuavaApp
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 0.742 s
[INFO] Finished at: 2026-03-24T22:00:09+05:30
[INFO]
[INFO] Killed gedit pom.xml (wd: ~/devops21/MyMavenApp)
[1]+  Killed gedit pom.xml (wd: ~/devops21/MyMavenApp)
[vaishnavi@vaishnavi-virtualbox:~/devops21]$
```

The command can be broken down as:

- DgroupId=com.example – project’s base package (reverse-domain style).
- DartifactId=mymavenguavaapp – defines the name of the project (will also be the folder name).
- DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.
- DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Use \$cd command to go into the directory.

\$gedit pom.xml

pom.xml file is created to configure the Maven project. It defines the project information, adds required dependencies like Guava and Commons IO libraries, and includes build plugins to compile the Java code, run unit tests, and package the application into an executable JAR file.

Add the dependencies:

```
<dependency>
  <groupId>com.google.guava</groupId>
```

```

<artifactId>guava</artifactId>

<version>33.5.0-jre</version>

<scope>compile</scope>
</dependency>
<!-- Source: https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>

  <groupId>commons-io</groupId>

  <artifactId>commons-io</artifactId>

  <version>2.21.0</version>

  <scope>compile</scope>
</dependency>

```

Execute \$tree command to view the structure of the project

```

vaishnav@vaishnavi-virtualbox:~/devops23$ cd myMavenGuavaApp1
vaishnav@vaishnavi-virtualbox:~/devops23/MyMavenGuavaApp1$ tree
.
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com
│   │   │       └── example
│   │   │           └── App.java
│   └── test
│       ├── java
│       │   └── com
│       │       └── example
│       │           └── AppTest.java
└── 9 directories, 3 files

```

Step 3:

Edit the App.java file

This **Step** opens the App.java file using the gedit editor. The code is pasted to create a Java program that uses the Google Guava library to create an immutable list and Apache Commons IO library to copy a file from source to destination

```
package com.example;
```

```
/**
```

```
 * Hello world!
```

```
 *
```

```
 */
```

```
// Import FileUtils class from Apache Commons IO library
```

```
// This is used to perform file operations like copying files import
```

```
org.apache.commons.io.FileUtils;
```

```
// Import File class to represent file and directory pathnames
```

```
import java.io.File;

// Import IOException to handle input-output errors import
java.io.IOException;

// Import ImmutableList from Google Guava library
// ImmutableList creates a list that cannot be modified after creation import
com.google.common.collect.ImmutableList;

// Define the main class public
class App
{
    // Main method - program execution starts from here public
    static void main( String[] args )
    {
        // Create an immutable list of strings (fruits)
        // This list cannot be changed (no add/remove allowed)
        ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana", "Cherry");

        // Print the list of fruits to the console
        System.out.println(fruits);

        // Create a File object representing the source file File
        sourceFile = new File("source.txt");

        // Create a File object representing the destination file File
        destFile = new File("destination.txt");

        try {
            // Copy the contents of source.txt to destination.txt
            // Using Apache Commons IO FileUtils class
            FileUtils.copyFile(sourceFile, destFile);

            // Print success message if file is copied
            System.out.println("File copied successfully!");
```

```

    }

    catch (IOException e) {

        // If any error occurs during file copying,

        // this block will handle the exception

        // Print error message

        System.err.println("Error occurred while copying file: " + e.getMessage());

    }

}

}

```

Step 4:

Execute the command `$mvn compile,test,package`

If the command shoes output as “build success”, it means that the project has compiled successfully

```

vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$ mvn compile
[INFO] Scanning for projects...
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenGuavaApp/src/main/resources
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenGuavaApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO] BUILD SUCCESS
[INFO] Total time: 0.476 s
[INFO] Finished at: 2026-03-24T22:57:20+05:30
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$

vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$ cd ..
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$ mvn test
[INFO] Scanning for projects...
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenGuavaApp/src/main/resources
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenGuavaApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenGuavaApp/src/test/resources
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenGuavaApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- maven-surefire-plugin:2.22.1:test (default-test) @ MyMavenGuavaApp ---
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO] Total time: 1.525 s
[INFO] Finished at: 2026-03-24T22:57:01+05:30
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$

```

Step 5:

Under the MyMavenGuavaApp project directory, create a file named source.txt and add some sample text such as “Hello Class”. This file will act as the source file that the Java program copies to destination.txt during execution.

```

[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$ mvn package
[INFO] Scanning for projects...
[INFO] -----
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] -----
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenGuavaApp/src/main/resources
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenGuavaApp ---
[INFO] nothing to compile - all classes are up to date
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenGuavaApp/src/test/resources
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenGuavaApp ---
[INFO] nothing to compile - all classes are up to date
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenGuavaApp ---
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in com.example.AppTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenGuavaApp ---
[INFO] BUILD SUCCESS
[INFO] Total time: 1.422 s
[INFO] Finished at: 2026-03-24T22:17:39+05:30
[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$

```

Step 6:

Execute `$ mvn exec:java -Dexec.mainClass="com.example.App"`

```

[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO] -----
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] -----
[INFO] --- exec-maven-plugin:1.6.3:java (default-cli) @ MyMavenGuavaApp ---
[INFO] Apple, Banana, Cherry
[INFO] File copied successfully!
[INFO] BUILD SUCCESS
[INFO] Total time: 0.456 s
[INFO] Finished at: 2026-03-24T23:00:43+05:30
[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$

```

Step 7:

After executing the program, the destination.txt file will be automatically created inside the MyMavenGuavaApp directory. The command `cat destination.txt` is used to display the contents of the file and verify the output.

```

[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$ ls
destination.txt  pom.xml  source.txt  src  target
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$ cat destination.txt
hiiiiiiiiiiiiiiii
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenGuavaApp$

```

Pushing The Project into Git:

Execute the commands:

```
$ git config --global user.name "vaishnavim1121"
$ git config --global user.email "vaishnavim1121@gmail.com"
$git init
$git add .
$git commit -m "initial commit"
$ git remote add origin https://github.com/vaishnavim1121/MyMavenGuavaApp.git
$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenGuavaApp
$ git push -u origin master
```

Second Phase

Depicting Jenkins Pipeline

Follow the given **Steps** to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on "New Item" to create a job, enter the item name select Pipeline as the project type.

✓ / New Item

New Item

Enter an item name

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project** (selected)
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

OK

This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click "OK" to proceed to the job configuration page

Now we need to add a jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins

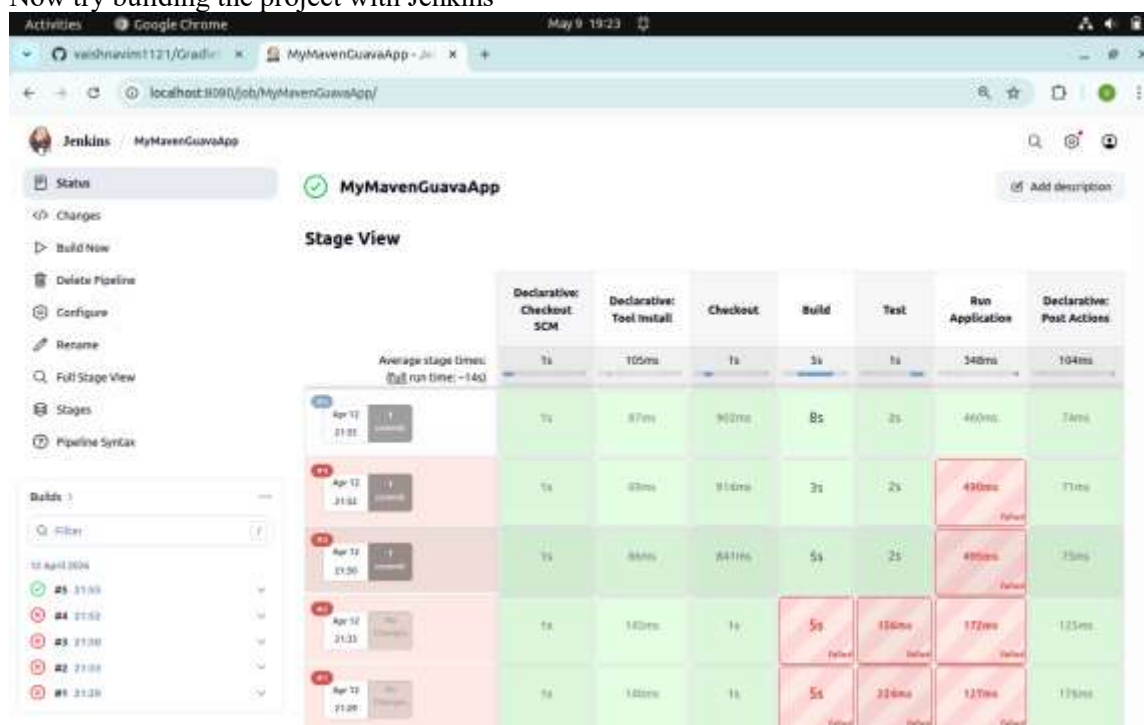
It includes **Step** wise instructions guiding at each stage with right commands

```

1 pipeline {
2   agent any // Use any available agent
3
4   tools {
5     maven 'Maven' // Ensure this matches the name configured in Jenkins
6   }
7   stages {
8     stage('Checkout') {
9       steps {
10        git branch: 'master', url: 'https://github.com/vaishnavini121/MyMavenApp.git'
11      }
12    }
13    stage('Build') {
14      steps {
15        sh 'mvn clean package' // Run Maven build
16      }
17    }
18    stage('Test') {
19      steps {
20        sh 'mvn test' // Run unit tests
21      }
22    }
23
24    stage('Run Application') {
25      steps {
26        // Start the JAR application
27        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
28      }
29    }
30  }
31 }
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Now try building the project with Jenkins



The error occurs because there is no Jenkinsfile in the github repository. Thus we need to push the content to our project repository in order to run the project successfully.

Push the project again to your git account

Now try building the project in Jenkins by clicking on build-now option.

And the project is built successfully.